DAY 11 — CLEAN ARCHITECTURE ENFORCEMENT + LOGGING SYSTEM

PART 1 — ARCHITECTURE ENFORCEMENT

Objective

Ensure the backend strictly follows **Clean Architecture principles**, making it:

- Maintainable
- Testable
- Scalable
- Industry-standard

Final Dependency Flow

```
API → Application → Domain
      ↓
  Infrastructure (via interfaces)
```

Key Principles

1. Dependency Rule

- Inner layers must NOT depend on outer layers
- Outer layers depend on inner layers

2. Separation of Concerns

Layer	Responsibility
API	HTTP, Controllers
Application	Business logic, CQRS
Domain (Core)	Entities, rules
Infrastructure	DB, external services

Implementation

1. Repository Pattern

Interface (Application Layer)

```
public interface IUserRepository
{
    Task AddAsync(User user, CancellationToken cancellationToken);
    Task<List<User>> GetAllAsync(CancellationToken cancellationToken);
    Task<bool> EmailExistsAsync(string email, CancellationToken cancellationToken);
    Task<User?> GetByEmailAsync(string email, CancellationToken cancellationToken);
}
```

Implementation (Infrastructure)

```
public class UserRepository : IUserRepository
{
    private readonly AppDbContext _context;

    public async Task<User?> GetByEmailAsync(string email, CancellationToken cancellationToken)
    {
        return await _context.Users
            .FirstOrDefaultAsync(u => u.Email == email, cancellationToken);
    }
}
```

2. CQRS with MediatR

Example: Login Handler

```
var user = await _userRepository.GetByEmailAsync(request.Email, cancellationToken);

if (user is null)
    throw new UnauthorizedException("Invalid email or password");
```

- No DbContext usage
- Only interface dependency

3. Dependency Injection

Moved ALL infra registrations to:
Infrastructure/DependencyInjection.cs

```
services.AddDbContext<AppDbContext>(...);
services.AddScoped<IUserRepository, UserRepository>();
services.AddScoped<IJwtTokenService, JwtTokenService>();
```

4. Removed Architectural Violations

- ❌ Removed duplicate DbContext registration
- ❌ Removed duplicate JWT service
- ❌ Removed duplicate repository methods
- ❌ Removed JwtTokenGenerator

IMPORTANT CONCEPT — Generics (ApiResponse<T>)

```
public class ApiResponse<T>
{
    public T? Data { get; set; }
}
```

T = placeholder type

Examples:

ApiResponse<Guid>
ApiResponse<List<UserDto>>

- Reusable
- Type-safe
- Clean API responses